Computer Vision

Lecture 10: Pose Estimation & Faces

Nguyen Manh Toan

Swinburne Vietnam

Week 10

Where We Left Off

Last Lectures

Detection learned to predict **boxes**, then to stop searching. Segmentation learned to predict **masks**, then to stop arguing about semantic versus instance.

Model predicts a *set*, matched to the truth by an assignment.

Today

Output is neither a box nor a mask.

Pose: a fixed number of named points, plus the edges between them.

Faces: not a label at all — a *position in a space*, because the answer at test time is an identity the model never saw.

The thread

Each of these fields made exactly one decisive move, and in both cases the move was **changing the output space**, not the architecture.

Today's Lecture

Pose

1. Why regressing coordinates fails, and what replaced it
2. Hourglass, SimpleBaseline, HRNet, ViTPose
3. The half-pixel, for the third time this semester
4. Many people: top-down against bottom-up
5. OKS, and why it is not a pixel threshold
6. Why 3-D is a different problem

Faces

7. The pipeline, and the one stage with a closed-form answer
8. Open set: why a classifier does not transfer
9. Normalise, then add a margin — three ways
10. Reading a verification number honestly
11. One threshold is not one policy

Pose: Changing the Output Space

The Output Is Neither a Box Nor a Mask

detection: 4 numbers



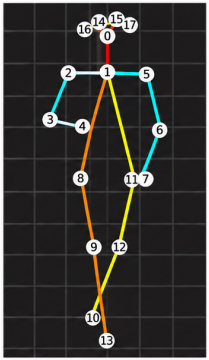
where, not how

segmentation: $H \times W$ labels



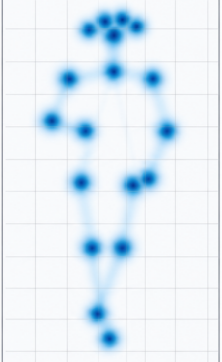
every pixel, no parts

pose: K points + a graph



17 points and what joins them

how it is actually predicted



K dense fields, one per joint

Pose is a structured output: the points are not interchangeable, and the edges carry as much information as the vertices. The *winning representation* is a field, not a list of coordinates.

The Obvious Thing

DeepPose (Toshev & Szegedy, 2014). A convolutional network, a fully connected head, and $2K$ outputs:

$$\mathcal{L} = \sum_{k=1}^{K} \left\| \hat{\mathrm{p}}_k - \mathrm{p}_k \right\|_2^2$$

Train on coordinates, predict coordinates, refine with a cascade.

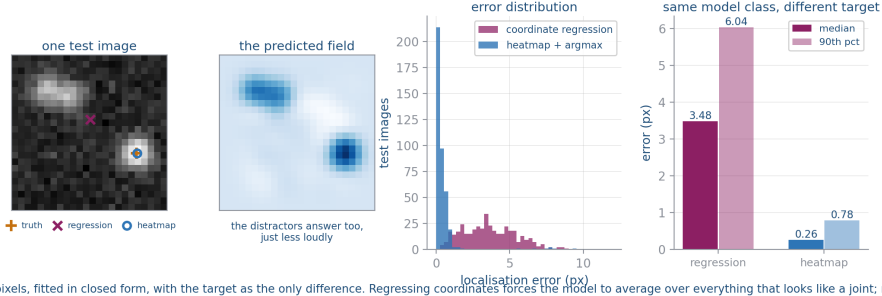
What it asks for

The last layer must turn a *spatial* feature map into *two numbers*. Every spatial position is pooled away, and the network has to re-encode position in the magnitudes of a vector.

A convolution is translation-equivariant. A fully connected regressor throws that property away at the last possible moment.

This is the same objection as Lecture 9's. The classifier's fully connected layer destroyed the spatial map; FCN's answer was to *not destroy it*.

The Same Experiment, Twice



One linear map from pixels, fitted in closed form, with the target as the only difference. Regressing coordinates forces the model to average over everything that looks like a joint; regressing a field lets each output pixel answer a local question and lets argmax arbitrate. This is why 2014 switched representations rather than architectures.

When three things in the image look like joints, a linear functional cannot pick one; it averages them. The prediction gets dragged toward the distractors.

The Reframe (Tompson et al., 2014)

Predict a field, not a number

Output K dense maps, one per joint. The target for joint k is a Gaussian centred on its true position:

$$H_k(\mathrm{u}) = \exp\left(-\frac{\|\mathrm{u} - \mathrm{p}_k\|^2}{2\sigma^2}\right)$$

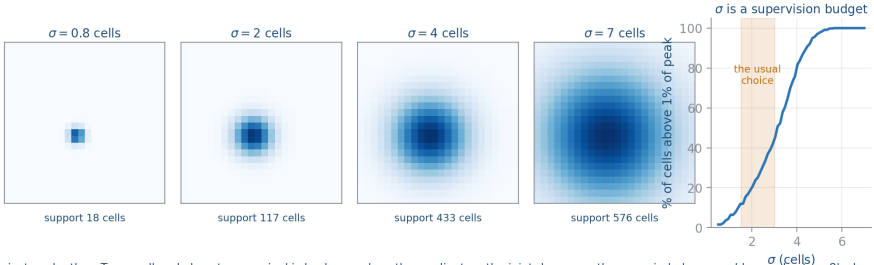
Train with per-pixel MSE. Read the answer back with arg max.

What that buys

- The loss is **local**: each pixel answers a question about its own neighbourhood.
- The architecture stays fully convolutional, so it stays translation-equivariant.
- The output is **multi-modal**. Two plausible elbows produce two peaks instead of their average.

When the output has spatial structure, predict something that has spatial structure.

The Target, and What σ Controls



σ trades two failures against each other. Too small and almost every pixel is background, so the gradient on the joint drowns --- the same imbalance problem as Lecture 8's dense heads. Too large and the peak is flat, so the argmax is decided by noise. Note the joint here sits at $(53.7, 41.2)$: whatever σ you pick, the grid cannot represent it.

What the Reframe Costs

Three new problems

1. The heatmap lives on a grid at stride s, so $\arg\max$ can only ever name a cell.
2. Almost every pixel is background — Lecture 8's imbalance.
3. Memory: K maps per person, not $2K$ numbers.

And one that stayed solved

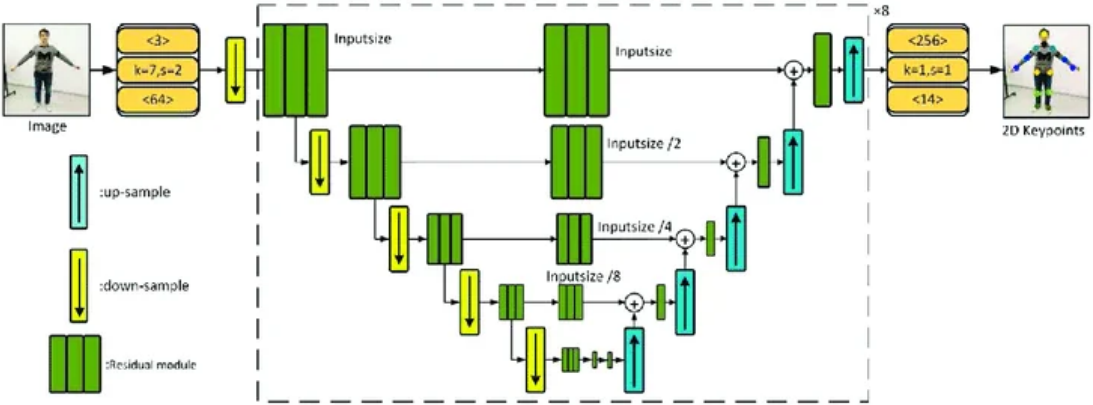
Regression never came back for single-person 2-D pose. Even *integral regression*, which takes the soft-argmax of a heatmap and so is trained on coordinates, keeps the heatmap in the middle.

Integral Pose Regression reaches 67.8 AP on COCO test-dev against SimpleBaseline's 73.7 — the field is a representation, not a detail of the loss.

Problem 1 is the one the field spent five years on, and we come back to it after the architectures.

Architectures for Keypoints

Stacked Hourglass (Newell et al., 2016)



SimpleBaseline (Xiao et al., 2018): How Little Is Enough

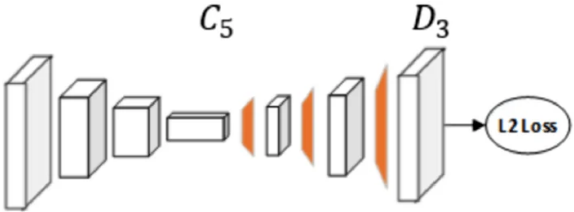
A ResNet, then **three transposed convolutions**, then a 1×1 convolution to K heatmaps. No skips, no stacking, no intermediate supervision, no multi-scale fusion.

It reached 73.7 AP on COCO test-dev — better than every hourglass variant of the time.

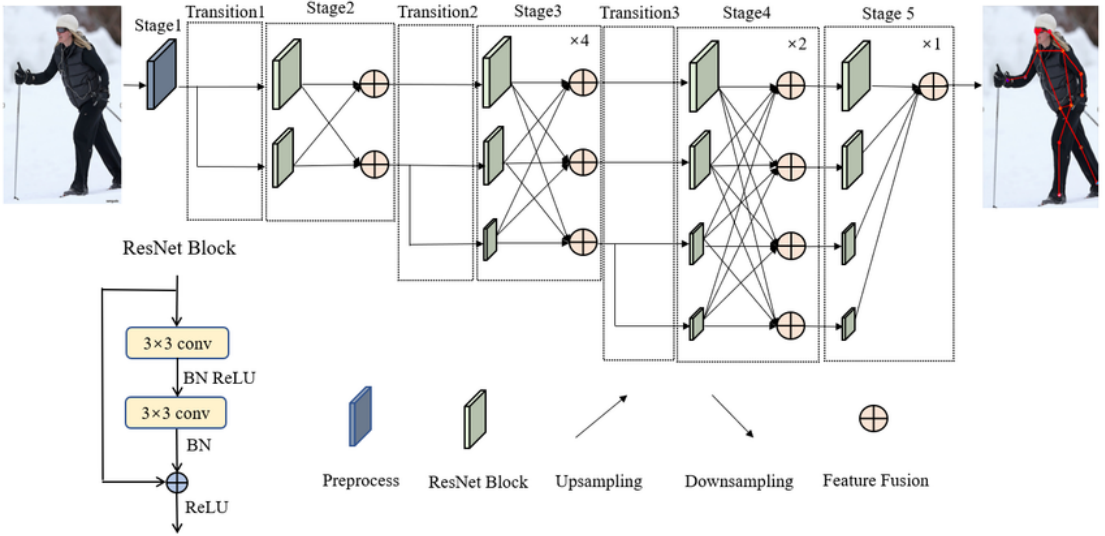
Why this paper matters

Not because the architecture was good. Because it established what the *baseline* was.

A great deal of 2016–2018 pose work turned out to be measuring better training schedules.



HRNet (Sun et al., 2019)



Why HRNet Was the Right Idea

Everything before it recovered

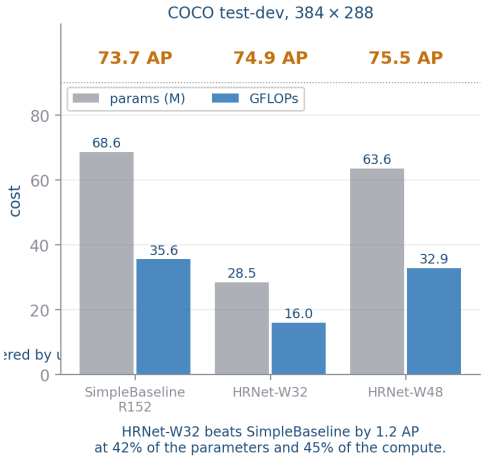
FCN, U-Net, hourglass, SimpleBaseline: downsample to get semantics, then try to undo it.

HRNet asks a different question — why lose it?

The cost

Keeping a stride-4 stream alive through the whole network, and the exchange units are fiddly to implement.

It is worth it here because a joint is a *point*, and a point is exactly the thing a coarse grid cannot represent.



HRNet then became a general backbone for segmentation, detection and classification, the usual sign that an idea was about representation rather than about one task.

ViTPose (Xu et al., 2022)

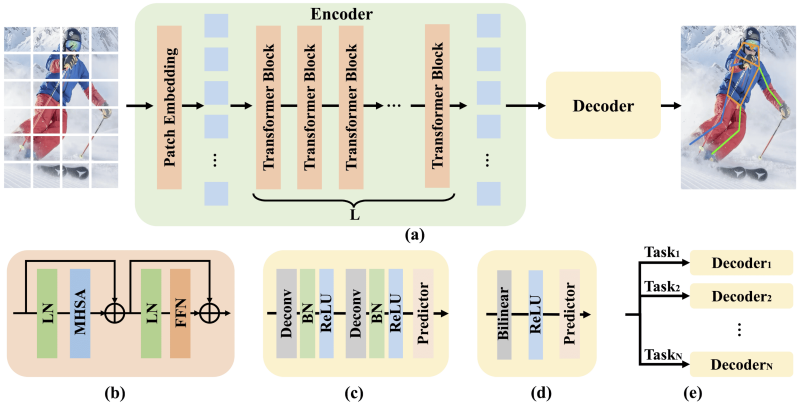


Figure: (a) The framework of ViTPose. (b) The Transformer Block. (c) The classic Decoder. (d) The simple Decoder. (e) The Decoder for multiple datasets.

ViTPose: The Backbone Stopped Mattering

A **plain** ViT — no pyramid, no multi-resolution streams — with a decoder of two deconvolutions, or even just *bilinear upsample, ReLU, one* 3×3 *convolution*. The simple decoder costs under 0.3 AP.

Scaled up, it reaches **80.9 AP** on COCO test-dev.

Read this carefully

It is not that HRNet was wrong. It is that at 1B parameters, pretrained at scale, the *inductive bias* HRNet hand-built stops being the binding constraint.

ViTPose-B is 86M parameters, ViTPose-H is 632M.

HRNet-W32 remains an excellent accuracy-per-FLOP choice, and the transformer results assume a pretraining budget we do not have.

More Than One Person

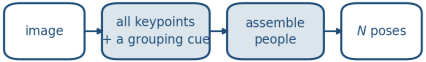
Two Pipelines

top-down



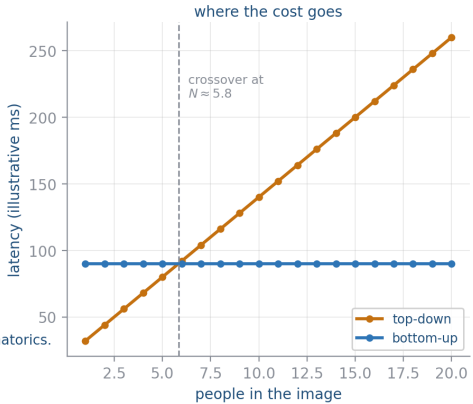
grouping is free; cost is linear in N; a missed box is a missed person

bottom-up



one pass whatever N is; grouping is now the hard part, and it is where the errors live

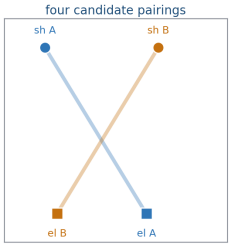
The two families do not differ in what they predict. They differ in where they put the combinatorics.

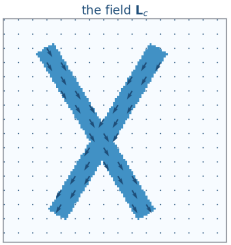


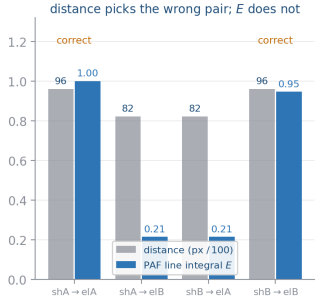
Shape, not measurements: the constants depend on your hardware, the crossover point does not go away.

OpenPose: Part Affinity Fields (Cao et al., 2017)

In "bottom-up" pose estimation, an AI model first detects all keypoints in an image. The challenge is connecting them correctly.





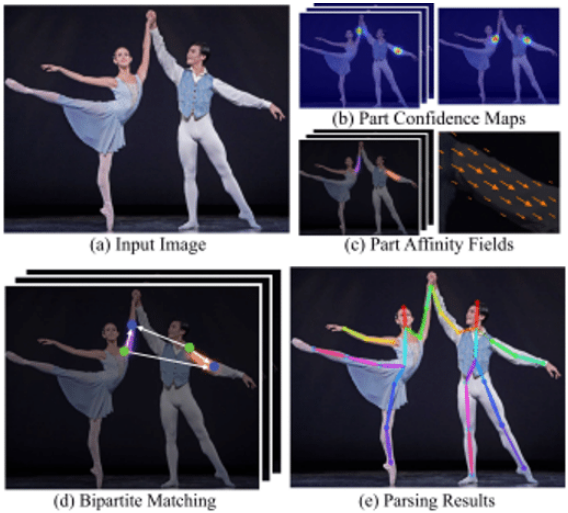


Shoulder A is nearer to elbow B than to its own elbow (82 px against 96 px), so any proximity rule groups the two people wrongly. The line integral does not care about distance: it asks whether the field along the segment points the way the segment does. Correct pairs score 1.00 and 0.95; crossed pairs score 0.21 and 0.21.

Instead of just looking at the joints, NN learns to predict the limbs connecting them.
A Part Affinity Field is a grid overlaid on the image where each pixel inside a limb contains an arrow.

OpenPose

- OpenPose tracks up to 135 keypoints per person in real time
- Use Part Affinity Fields to associate body parts with specific individuals



The Line Integral

For two candidate joints d_{j_1} and d_{j_2}, the association score is the field's agreement with the segment joining them:

$$E = \int_{u=0}^{1} \mathrm{L}_c\big(\mathrm{p}(u)\big) \cdot \frac{\mathrm{d}_{j_2} - \mathrm{d}_{j_1}}{\|\mathrm{d}_{j_2} - \mathrm{d}_{j_1}\|_2}\, du, \qquad \mathrm{p}(u) = (1-u)\,\mathrm{d}_{j_1} + u\,\mathrm{d}_{j_2}$$

evaluated by sampling u uniformly.

Why a vector field and not a score map

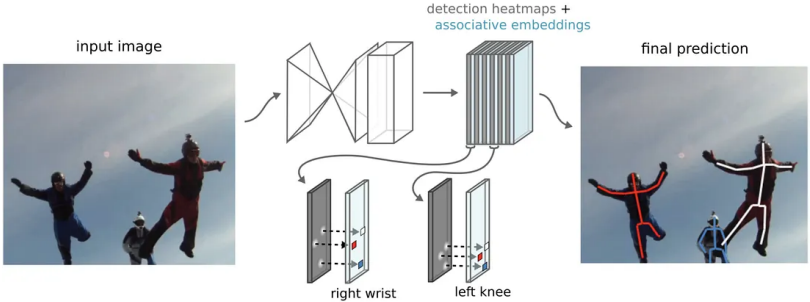
A scalar "limb-ness" map cannot distinguish an arm from the arm beside it crossing the same pixels. The **direction** can.

The assembly is approximate

Assembling full bodies from pairwise scores is a K-dimensional matching problem, which is NP-hard. OpenPose relaxes it to a sequence of bipartite matchings and takes the greedy answer.

It works because the network's receptive field is large enough that the pairwise scores already carry global context.

Associative Embedding (Newell et al., 2017)



Single-stage pipeline

- **Detection Heatmaps**: outputs per-pixel (or per-joint) detection scores to locate candidate parts
- **Tag/Embedding Heatmaps**: outputs an identity "tag" (a real-valued embedding) for each detection
- **Grouping by Distance**: Detections that belong to the same person or object have similar tag values.

Which Family Should You Use?

Top-down

75.5 AP (HRNet-W48, COCO test-dev).

- Each person is cropped and resized, so small people get full resolution.
- Cost grows with the number of people.
- A person the detector missed is simply gone.

Bottom-up

70.5 AP (HigherHRNet, COCO test-dev).

- One pass whatever the crowd size.
- Small people stay small — they are never zoomed in on.
- Grouping errors are a failure mode top-down simply does not have.

The five-point gap is mostly about scale, not grouping

Crop-and-resize is a free super-resolution trick, and it is the single biggest reason top-down still leads on a benchmark full of small people.

Measuring a Pose

OKS: IoU for Points

Object Keypoint Similarity (OKS) acts as the IoU-analog for human pose and keypoint estimation.

Measures how close each predicted keypoint is to its ground-truth location using an un-normalized Gaussian distribution

$$\text{OKS} = \frac{\sum_i \exp\left(-\frac{d_i^2}{2s^2k_i^2}\right)\delta(v_i > 0)}{\sum_i \delta(v_i > 0)}$$

- d_i: Euclidean distance between the predicted keypoint and the ground-truth keypoint
- s: Object scale (the square root of the object's segmentation area), making the metric scale-invariant
- k_i: Per-keypoint constant ($k_i = 2\sigma_i$) which controls the tolerance for each specific joint
- v_i: Visibility flag for keypoint i (used to filter out unlabeled or occluded points)

OKS computes the average score across all visible labeled keypoints, resulting in a score from 0.0 to 1.0

Where the Benchmarks Are

MPII, PCKh@0.5

Wei et al. (2016)	88.5
Newell et al. (hourglass)	90.9
SimpleBaseline	91.5
HRNet-W32	92.3

Four points in four years. This benchmark is finished.

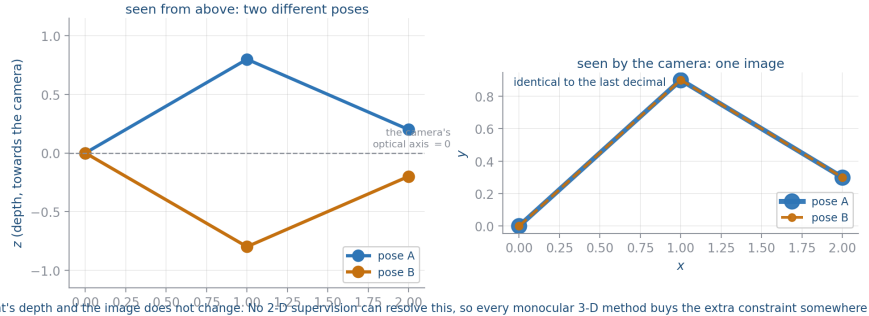
COCO keypoints, test-dev AP

Mask R-CNN	63.1
CPN	72.1
SimpleBaseline (R152)	73.7
HRNet-W48	75.5
ViTPose (ViTAE-G)	80.9

Still moving — because OKS is a sweep over thresholds and the strict end is far from saturated.

A benchmark that stops moving has not been solved; it has stopped being able to tell two methods apart. MPII's remaining errors are largely annotation disagreements.

Depth Is Not in the Image



Flip a joint's depth and the image does not change. No 2-D supervision can resolve this, so every monocular 3-D method buys the extra constraint somewhere else: a learned prior over plausible human poses, a parametric body model with limb lengths fixed, temporal smoothness, or a second camera. When you read a 3-D pose result, the first question is which of those it used.

So Where Does 3-D Come From?

Four ways to buy the constraint

- **A learned pose prior.** Most 3-D poses are not anatomically possible; train on the ones that are.
- **A body model** (SMPL and relatives). Fix limb lengths and predict pose parameters, not points.
- **Time.** A smooth trajectory rules out a flip that would be instantaneous.
- **A second camera.** The only one that actually measures depth.

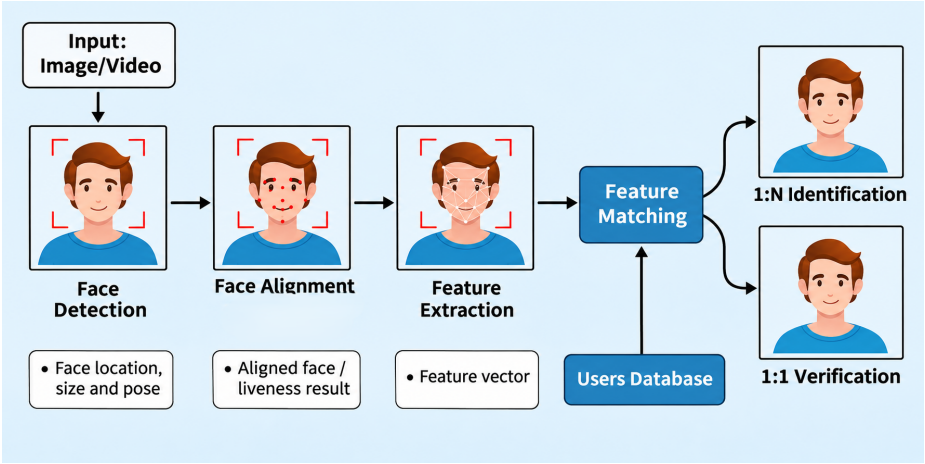
Reading a 3-D result

The first question is always which of the four it used, and the second is whether the test set contains poses the prior has never seen.

A monocular 3-D system evaluated only on the dataset its prior was trained on is reporting how well it memorised a pose distribution.

Faces: A Different Output Space

The Pipeline



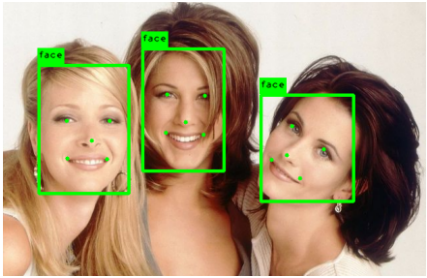
The last blue box makes face recognition different from classification. At test time, the identities are ones the model never saw, so there is no classifier to apply, only a distance. Everything in the training loss exists to make that distance mean something.

Detection: Faces Are Not Just Small Objects

RetinaFace (Deng et al., 2019) trains one single-stage detector on four losses at once:

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \lambda_1 \mathcal{L}_{\text{box}} + \lambda_2 \mathcal{L}_{\text{pts}} + \lambda_3 \mathcal{L}_{\text{pixel}}$$

classification, box, **five landmarks**, and a dense 3-D surface regression. The last three applied only where there is a face.



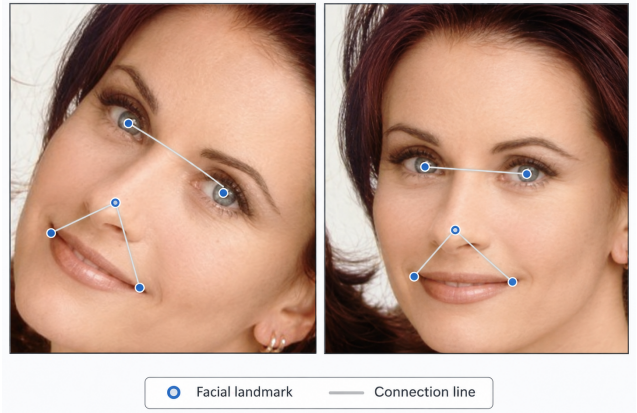
Why landmarks in the detector

Because the next stage needs them anyway. Predicting them here is free supervision — an extra signal about where the face *is* that the box alone does not give.

91.4 AP on WIDER FACE *hard*, against 90.3 for the best two-stage method of the time.

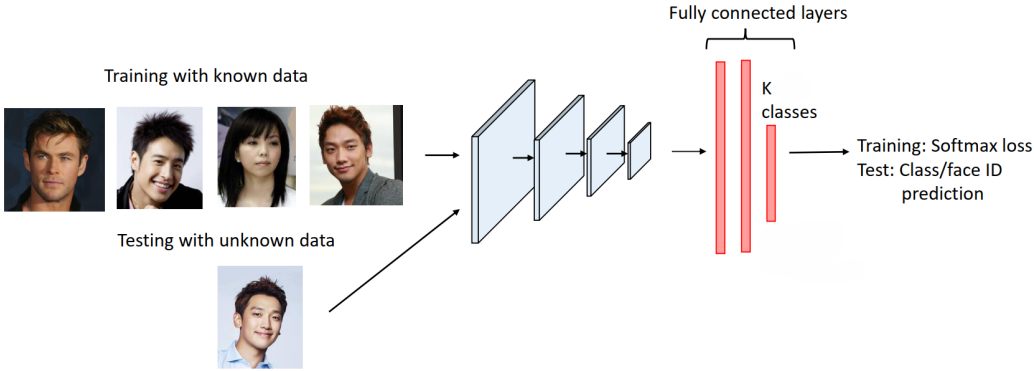
Alignment: The One Stage With a Closed-Form Answer

Every other stage in the face pipeline is learned and approximate. Alignment is the exception.



Alignment typically acts as a preprocessing step designed to normalize scale, rotation, and translation variations by localizing facial landmarks.

Classification

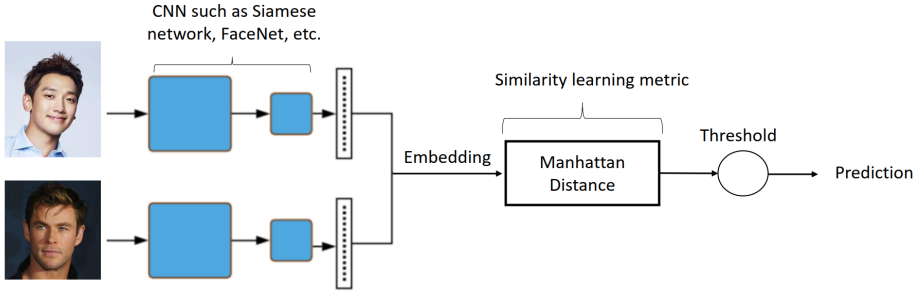


Train on K classes, test on the same K classes. The final layer *is* the answer.

Recognition: The Set Is Open

Train on 100k celebrities; deploy on your company's 400 employees, none of whom were in the training set.

The final layer is **thrown away**. What ships is the layer before it, and a distance.



The consequence

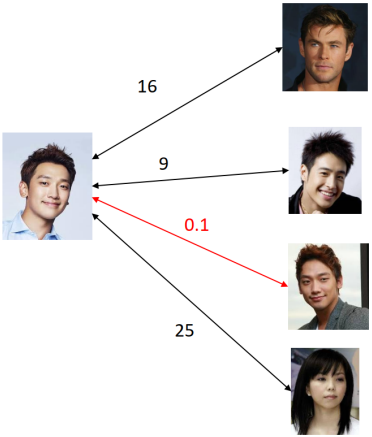
Training-set accuracy is *not* the objective. The objective is that the embedding's geometry generalises to identities the loss never mentioned.

Similarity Function

$d(I_1, I_2) = \|f(I_1) - f(I_2)\|$ is the degree of difference between two images I_1, I_2.
For face verification:

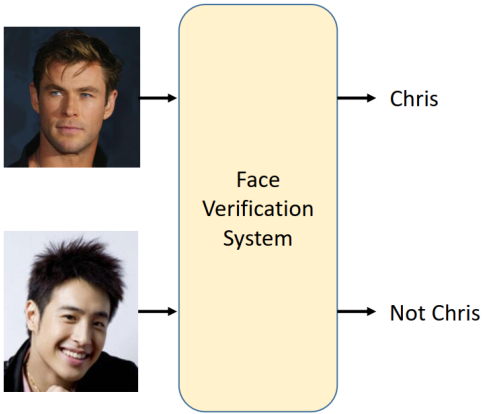
- If $d(I_1, I_2) \leq \tau$, then two images are the same.
- If $d(I_1, I_2) > \tau$, then two images are different.

For face recognition, similarity function is used to check for every pairs of the images.



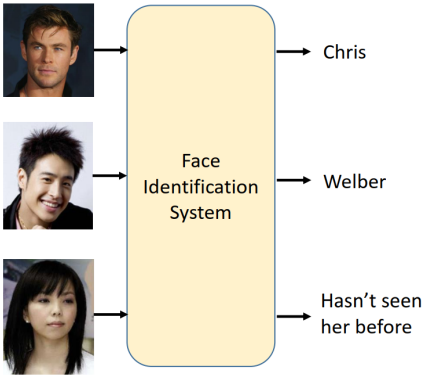
Verification (1:1)

- One comparison, one threshold.
- The false accept rate is the probability that *this* impostor pair passes.
- **Applications**: Unlocking a phone; matching a passport to its holder.



Identification (1:N)

- N comparisons, and accept the best.
- With a gallery of N, the chance of *some* impostor passing grows roughly like $N\times$ FAR.
- A watchlist of 10^5 at FAR 10^{-5} gives a false alarm on **about every face that walks past**.



A system that is excellent at verification can be useless at identification.

FaceNet (Schroff et al., 2015): Learn the Distance Directly



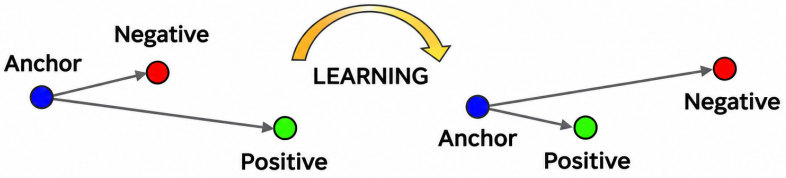
Anchor



Positive



Negative



Triplet loss: $\max(\|f_a - f_p\| - \|f_a - f_n\| + m, 0)$

Triplet Mining

With N images there are $O(N^3)$ triplets, and almost all of them are already satisfied and contribute zero gradient.

So the method becomes a **mining** problem: which triplets to look at

- Easy triplets: $\|f_a - f_p\| + m \leq \|f_a - f_n\|$.
- Hard triplets: $\|f_a - f_n\| \leq \|f_a - f_p\|$.
- Semi-hard triplets: $\|f_a - f_p\| \leq \|f_a - f_n\| \leq \|f_a - f_p\| + m$.

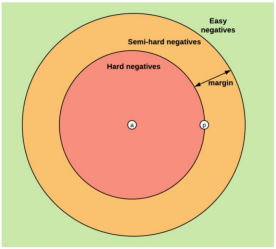


Figure: Regions of embedding space for negatives.

The move that followed

Keep the softmax classifier — with its one gradient per sample and no sampling decisions — and change its *geometry* until it produces the embedding you wanted.

Angular Margin Losses

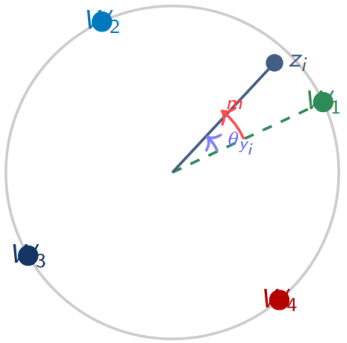
Enforce margins on the *hypersphere* using learnable class centres W_j.
The logit for class j becomes $s\cos\theta_j$:

Method	Margin type	Score
SphereFace (2017)	Multiplicative	$\cos(m\theta)$
CosFace (2018)	Additive cosine	$\cos\theta - m$
ArcFace (2019)	Additive angular	$\cos(\theta + m)$

Highly effective for both closed-set and open-set tasks

No sampled negatives

Repulsion comes from all $C-1$ class centres simultaneously — no negative mining needed.

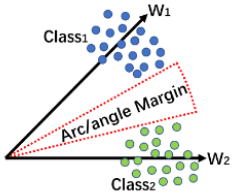


unit hypersphere

Additive Angular Margin Loss (Deng et al., 2019 - ArcFace)

$$\mathcal{L}_{\text{Arc}} = -\frac{1}{N}\sum_{i=1}^{N}\log\frac{e^{s\cos(\theta_{y_i}+m)}}{e^{s\cos(\theta_{y_i}+m)}+\sum_{j\neq y_i}e^{s\cos\theta_j}}$$

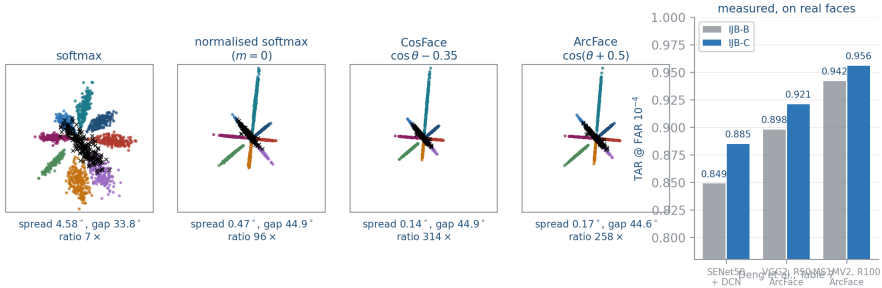
- $\theta_{y_i} = \arccos\left(W_{y_i}^{\top} z_i\right)$: angle between embedding z_i and class centre W_{y_i}
- $m > 0$: additive angular margin (typically $m = 0.5$): simultaneously enhance the intra-class compactness and inter-class discrepancy
- $s > 0$: feature scale (typically $s = 64$) -radius of the hypersphere
- Enforces a hard margin against **all** $C-1$ other classes simultaneously



Why a Margin at All

Without one: Cross-entropy stops pushing as soon as the right class wins. A sample sitting just inside its own decision region produces almost no gradient.

With one: The sample must beat the others *by* m. Correct is no longer good enough, so the optimizer keeps compressing each class until it has m to spare.



Eight training identities in colour, one the model never saw in black, in a two-dimensional feature layer so that the geometry can be drawn. Plain softmax separates classes by magnitude as much as by angle, which is the wrong currency for a cosine threshold. Normalising fixes that; the margin then makes each class an order of magnitude tighter relative to the gap between them ($7\times \to 96\times \to 314\times$). Two dimensions cannot hold enough identities to test the open-set claim, so the last panel does that with published numbers instead of a toy.

The Numbers, and How Much They Say

Loss	Margin	LFW	CFP-FP	AgeDB-30
Norm-Softmax	—	98.56	90.67	89.50
Softmax	—	99.08	93.81	90.92
Triplet	0.35	98.98	95.25	94.55
SphereFace	$m_1 = 1.35$	99.11	95.44	94.56
CosFace	$m_3 = 0.35$	99.51	95.24	94.86
ArcFace	$m_2 = 0.5$	99.53	95.56	95.15
combined	$(0.9, 0.4, 0.15)$	99.50	89.79	88.72

All rows: CASIA, ResNet50, same training recipe (Deng et al., Table 2).

LFW Dataset

99.08 to 99.53. Forty-five hundredths of a point separates a bad loss from the best one. **LFW cannot measure this any more.**

Margin Combination

Combining margins is not automatically better. $(0.9, 0.4, 0.15)$ loses six points on CFP-FP while looking fine on LFW.

Reading a Face-Recognition Number

Verification Is Not Accuracy



A single accuracy number hides all of this. The operating point you care about is set by how often the system is used: a phone unlocking a thousand times a year and a border gate seeing a million people a year need thresholds three orders of magnitude apart. And a benchmark with three thousand impostor pairs cannot report a false accept rate below one in three thousand, however many decimal places its accuracy is quoted to.

Verification Metrics

A pair is **genuine** (same person) or **impostor** (not). One score s, one threshold τ, accept when $s \geq \tau$. That gives exactly two error rates:

$$\text{False Match Rate}$$
$$\text{FMR}(\tau) = \Pr(s \geq \tau \mid \text{impostor})$$
$$\text{False Non-Match Rate}$$
$$\text{FNMR}(\tau) = \Pr(s < \tau \mid \text{genuine})$$

and $\text{TAR} = 1 - \text{FNMR}$. Moving τ trades one against the other: **one knob, two errors**.

The summaries, and what each hides

- **DET**: FNMR against FMR, both log — on a linear ROC the region you care about is the left edge.
- **EER**, where FMR = FNMR: no threshold to state, and almost never your operating point.
- **TAR at a stated FAR**: the number to report, useless without the "at".
- **Accuracy**: assumes 50:50 pairs.

	EER	**TAR @ FAR** 10^{-4}	**TAR @ FAR** 10^{-6}
model A	0.56%	97.28%	91.73%
model B	0.38%	95.43%	81.10%

[computed] B wins on EER; its 0.4% look-alike impostors show up only in the tail. **EER ranks these two backwards.**

The Benchmark That Can Still Tell Methods Apart

Method	IJB-B	IJB-C
ResNet50	0.784	0.825
SENet50 + DCN	0.849	0.885
ArcFace (VGG2, R50)	0.898	0.921
ArcFace (MS1MV2, R100)	0.942	0.956

TAR at FAR $= 10^{-4}$ (Deng et al., Table 7).

Compare the two tables

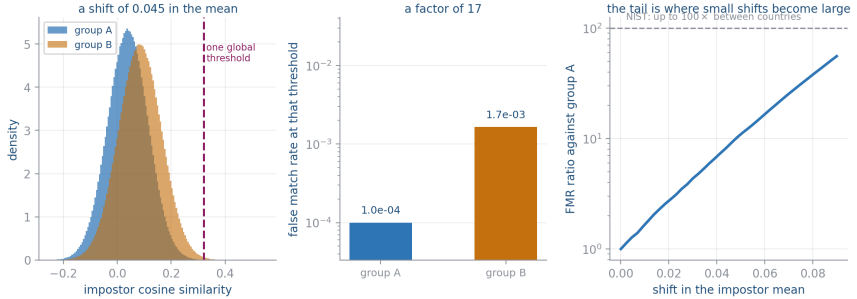
On LFW these methods span 0.45 points. Here they span **16**.

Nothing changed about the models. The benchmark simply has enough impostor pairs to ask a harder question.

What to do in your own work

Report TAR at a stated FAR, and say how many impostor pairs the number came from. "99.4% accuracy" on a few thousand pairs is not a result; it is a confidence interval wearing a disguise.

One Threshold Is Not One Policy



The mechanism, computed; the measurements are NIST's. A difference too small to see in the histogram becomes a large difference in false matches, because a threshold set for FAR 10^{-4} sits far out in the tail where the density is falling fast. NISTIR 8280 found false positives highest for West and East African and East Asian faces and lowest for Eastern European, "a factor of 100 more false positives between countries"; 2 to 5 times higher for women than men; and elevated for the elderly and for children. One global threshold is therefore not one policy.

Left: Two impostor score distributions (non-matching pairs) whose means differ by 0.045 in cosine similarity. One global threshold set so group A hits FAR $= 10^{-4}$.
Middle: Group A false-matches at 1.0×10^{-4} and group B at 1.7×10^{-3}. A factor of 17.
Right: The small became large.

NISTIR 8280 Report

NIST evaluated 189 algorithms from 99 different developers, analyzing 18.27 million images across 8.49 million unique subjects on government-held datasets.

- False positives highest for **West and East African and East Asian** faces, lowest for **Eastern European**, with "a factor of 100 more false positives between countries".
- **2 to 5 times** higher for women than men, consistently across algorithms and datasets.
- Elevated for the **elderly** and for **children**.

Two details worth keeping

- Some algorithms developed in China showed the *opposite* pattern — low false positives on East Asian faces. So this is about training data, not about faces.
- False *positives* are the asymmetric error: a false negative means try again; a false positive means someone is accused.

MS-Celeb-1M

Ten million images of 100,000 people, scraped from the web without consent. It trained most of the results on the previous slides.

Microsoft withdrew it in June 2019. The copies did not disappear, and papers using it kept appearing for years.



For Your Project

Choosing: Pose

If	Use	Why
one person, accuracy matters	top-down HRNet-W32	best AP per FLOP
a crowd, fixed latency budget	bottom-up (HigherHRNet)	cost independent of N
edge device, real time	a small top-down model	crop is cheap, backbone is not
a custom keypoint set	SimpleBaseline first	establish the baseline
you need 3-D	a body model, or two cameras	2-D cannot resolve depth

Whatever you choose, do this first

Check your heatmap encoding and decoding. It is worth more AP than your choice of backbone and it costs an afternoon.

Choosing: Faces

Almost always

A pretrained ArcFace embedding, frozen. Fine-tuning on a few hundred identities will usually make it worse, because the margin loss needs many identities to shape the space.

Spend your effort on detection, alignment and the threshold instead.

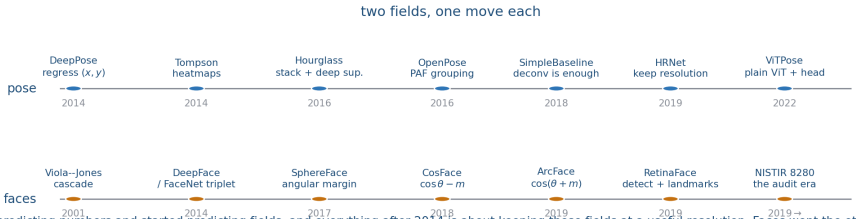
Set the threshold on your own data

The published threshold was tuned on a benchmark whose impostor distribution is not yours.

Collect impostor pairs from *your* population, pick the FAR your application can live with, and report the TAR you get — per demographic group if you have the labels.

And if the application is surveillance or policing, the technical questions on this slide are not the first ones to ask.

The Shape of Both Arguments



Pose stopped predicting numbers and started predicting fields, and everything after 2014 is about keeping those fields at a useful resolution. Faces went the other way: they stopped treating identity as a class to predict and started shaping the space the features live in, so that an identity never seen in training still has somewhere to be. Both are the same lesson --- choose the output space first, then the architecture.

Summary

Pose

- Coordinates were the obvious output and the wrong one; a dense field is translation-equivariant, local and multi-modal.
- Every architecture since is about keeping that field at a usable resolution — hourglass recovers it, HRNet never loses it.
- The encode/decode half-pixel is worth more than most architectural choices.
- OKS is calibrated to human annotator disagreement, per joint.

Faces

- The test identities are not the training identities, so the classifier is discarded and only the geometry ships.
- Normalise away everything the cosine cannot see, then add a margin so that "correct" is not good enough.
- Accuracy on LFW stopped being a measurement around 2017; report TAR at a stated FAR.
- One global threshold produces different error rates for different people, and NIST has measured by how much.

Reading & Reference

Pose

- Toshev & Szegedy, *DeepPose*, CVPR 2014
- Tompson et al., *Joint Training of a CNN and a Graphical Model*, NIPS 2014
- Newell et al., *Stacked Hourglass Networks*, ECCV 2016
- Cao et al., *Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields*, CVPR 2017 / TPAMI 2019
- Newell et al., *Associative Embedding*, NIPS 2017
- Xiao et al., *Simple Baselines*, ECCV 2018
- Sun et al., *Deep High-Resolution Representation Learning*, CVPR 2019
- Zhang et al., *Distribution-Aware Coordinate Representation* (DARK), CVPR 2020
- Cheng et al., *HigherHRNet*, CVPR 2020
- Xu et al., *ViTPose*, NeurIPS 2022

Faces

- Schroff et al., *FaceNet*, CVPR 2015
- Liu et al., *SphereFace*, CVPR 2017
- Wang et al., *CosFace*, CVPR 2018
- Deng et al., *ArcFace*, CVPR 2019
- Deng et al., *RetinaFace*, CVPR 2020
- Grother, Ngan & Hanaoka, *FRVT Part 3: Demographic Effects*, NISTIR 8280, 2019

Code

- `mmpose` — every pose model above, one config each
- `insightface` — ArcFace, RetinaFace, and the 112×112 alignment template used in this lecture
- `pycocotools` — `cocoeval.py` holds the σ_i values and the OKS implementation

Hands-On

`lecture10_handson.ipynb`: the representation, then the geometry.

1. Reproduce the regression-against-heatmap experiment, then break it: find the regime where regression wins
2. Encode and decode a sub-pixel keypoint; measure the $s\sqrt{2/12}$ floor and escape it
3. OKS from the published σ_i, and $\mathrm{AP}^{\mathrm{OKS}}$ end to end
4. Part affinity fields: build the field, score the line integral, break the greedy assembly
5. Associative embedding's grouping loss, by gradient descent
6. Umeyama alignment, and what misalignment costs an embedding
7. Implement all four margin losses and plot the target logits
8. Verification: ROC, TAR at FAR, and the per-group threshold experiment from the last section